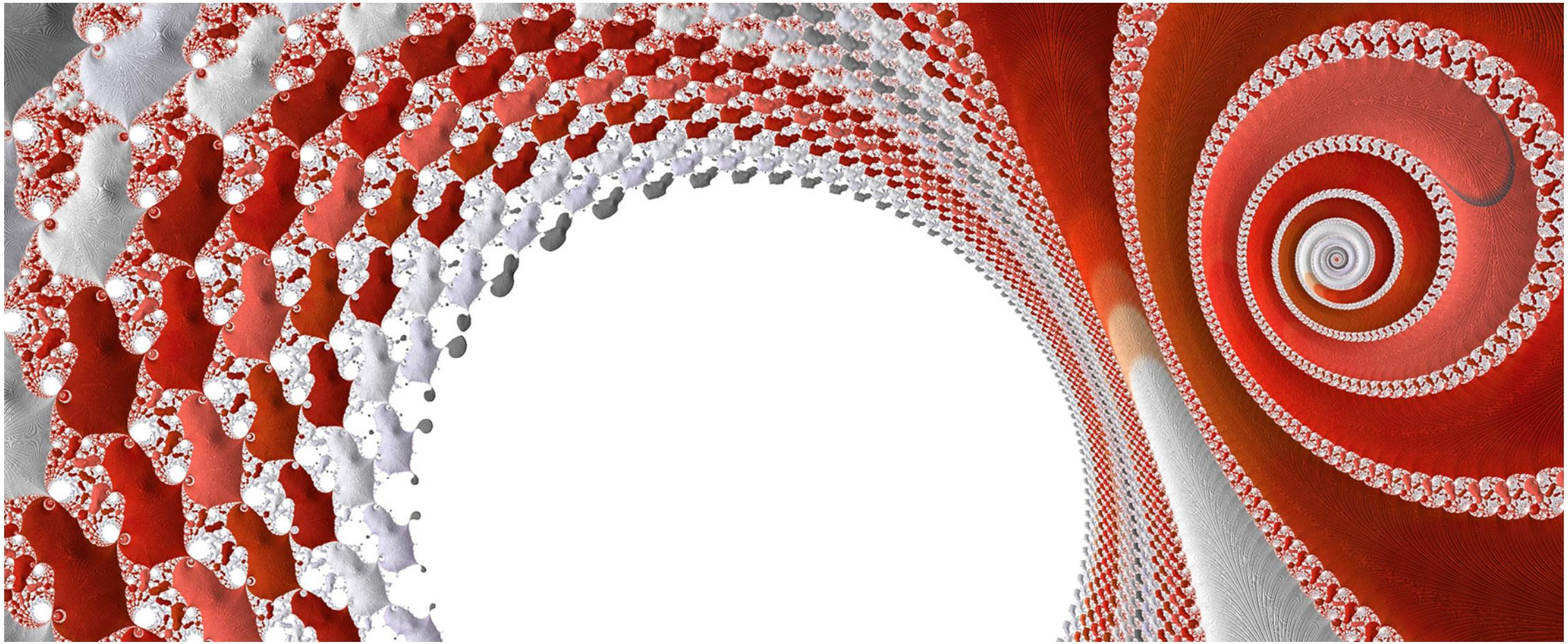
DS-011 Foundations for Data Science in Python

Access for free at https://openstax.org/details/books/introduction-python-programming

Module 5 - Loops



Credit: modification of work "Quantum Computing", by Kevin Dooley/Flickr, CC BY 2.0

A loop is a code block that runs a set of statements while a given condition is true. A loop is often used for performing a repeating task.

Ex: The software on a phone repeatedly checks to see if the phone is idle. Once the time set by a user is reached, the phone is locked. Loops can also be used for iterating over lists like student names in a roster, and printing the names one at a time.

In this chapter, two types of loops, `for` loop and `while` loop, are introduced. This chapter also introduces break and continue statements for controlling a loop's execution.

5.1 While Loop

Learning objectives

By the end of this section you should be able to

- Explain the loop construct in Python.
- Use a `while` loop to implement repeating tasks.

While loop

A **while loop** is a code construct that runs a set of statements, known as the loop body, while a given condition, known as the loop expression, is true. At each iteration, once the loop statement is executed, the loop expression is evaluated again.

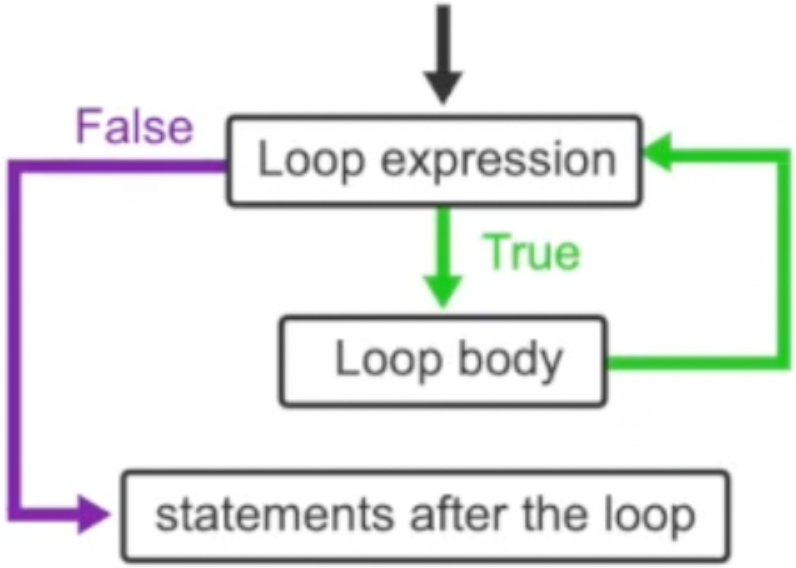
- If true, the loop body will execute at least one more time (also called looping or iterating one more time).
- If false, the loop's execution will terminate and the next statement after the loop body will execute.

While loop

Template:

```
# intialization
while expression:
  # loop body

# statements after the loop
```



Example:

```
total = 0
while total<= 10:
    n = int(input("Enter number:"))
    total += n
    print("total=", total)
print("End of program")
```

Quiz

- Fibonacci is a series of numbers in which each number is the sum of the two preceding numbers. The Fibonacci sequence starts with two ones: 1, 1, 2, 3, ...
- Consider the following code that prints all Fibonacci numbers less than 20, and answer the following questions.

```
# Initializing the first two Fibonacci numbers
f = 1
g = 1
print (f, end = ' ')

# Running the loop while the last Fibonacci number is less than 20
while g < 20:
    print(g, end = ' ')
    # Calculating the next Fibonacci number and updating the last two sequence numbers
    temp = f
    f = g
    g = temp + g
```

How many times does the loop execute?

- [ ] 5
- [ ] 6
- [ ] 7

How many times does the loop execute?

- [ ] 5
- [x] 6
- [ ] 7

What is the variable g's value when the while loop condition evaluates to False?

- [ ] 13
- [ ] 20
- [ ] 21

What is the variable g's value when the while loop condition evaluates to False?

- [ ] 13
- [ ] 20
- [x] 21

What are the printed values in the output?

- [ ] 1 1 2 3 5 8 13
- [ ] 1 2 3 5 8 13
- [ ] 1 1 2 3 5 8 13 21

What are the printed values in the output?

- [x] 1 1 2 3 5 8 13
- [ ] 1 2 3 5 8 13
- [ ] 1 1 2 3 5 8 13 21

Counting with a while loop

- A `while` loop can be used to count up or down.
- A counter variable can be used in the loop expression to determine the number of iterations executed.
- Ex: A programmer may want to print all even numbers between 1 and 20.
  - The task can be done by using a counter initialized with 1.
  - In each iteration, the counter's value is increased by one, and a condition can check whether the counter's value is an even number or not.
  - The change in the counter's value in each iteration is called the **step size**.
  - The step size can be any positive or negative value.
  - If the step size is a positive number, the counter counts in ascending order, and if the step size is a negative number, the counter counts in descending order.

Example: Printing all odd numbers between 1 and 10

```
# Initialization
counter = 1

# While loop condition
while counter <= 10:
  if counter % 2 == 1:
    print(counter)
  # Counting up and increasing counter's value by 1 in each iteration
  counter += 1
```

Example: Counting with while loop

```
# Initialization
counter = 5

# While loop condition
while counter > 0:
    print(counter * '*')
    # Decreasing counter's value by 1 in each iteration
    counter -= 1
```

Quiz

Given the code, answer the following questions.

```
n = 4
while n > 0:
    print(n)
    n = n - 1

print("value of n after the loop is", n)
```

How many times does the loop execute?

- [ ] 3
- [ ] 4
- [ ] 5

Given the code, answer the following questions.

```
n = 4
while n > 0:
    print(n)
    n = n - 1

print("value of n after the loop is", n)
```

How many times does the loop execute?

- [ ] 3
- [x] 4
- [ ] 5

Which line is printed as the last line of output?

- value of n after the loop is -1.
- value of n after the loop is 0.
- value of n after the loop is 1.

Which line is printed as the last line of output?

- [ ] value of n after the loop is -1.
- [x] value of n after the loop is 0.
- [ ] value of n after the loop is 1.

What happens if the code is changed as follows?

```
n = 4
while n > 0:
  print(n)
  # Modified line
  n = n + 1

print("value of n after the loop is", n)
```

- The code will not run.
- The code will run for one additional iteration.
- The code will never terminate.

What happens if the code is changed as follows?

```
n = 4
while n > 0:
  print(n)
  # Modified line
  n = n + 1

print("value of n after the loop is", n)
```

- [ ] The code will not run.
- [ ] The code will run for one additional iteration.
- [x] The code will never terminate.

TRY IT

5.1 Reading inputs in a while loop

TRY IT

5.1 Sum of odd numbers

5.2 For Loop

Learning objectives

By the end of this section you should be able to

- Explain the `for` loop construct.
- Use a `for` loop to implement repeating tasks.

For loop

- In Python, a **container** can be a range of numbers, a string of characters, or a list of values. To access objects within a container, an iterative loop can be designed to retrieve objects one at a time.
- A **for loop** iterates over all elements in a container.

  Ex: Iterating over a class roster and printing students' names.

Iterating over a container

Template:

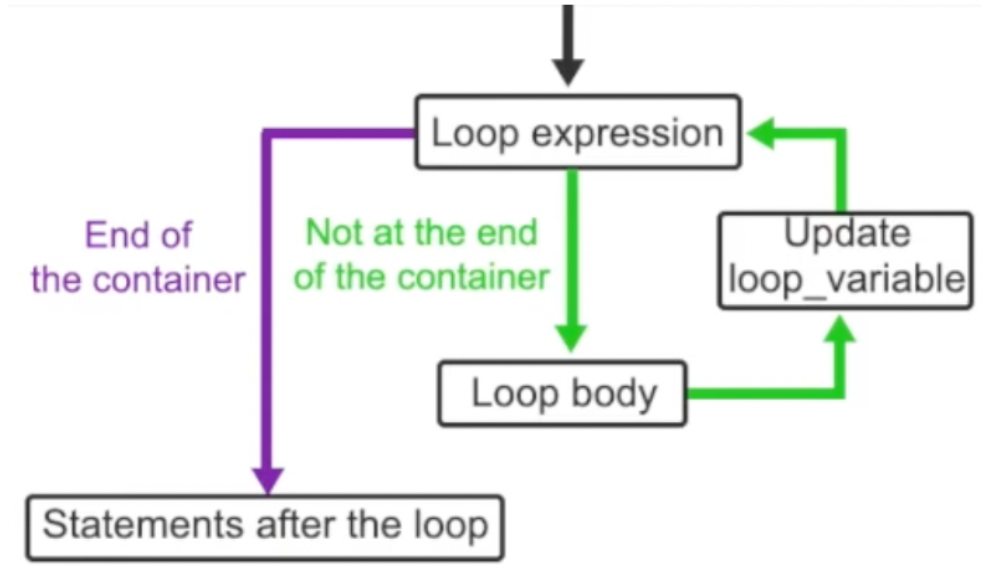
```
# intialization
for loop_variable in container:
  # loop body

# statements after the loop
```

Example:

```
names_list = ['Sara', 'Ben', 'Jaya']

for name in names_list:
    print("Hi", name)
```



Quiz

A string variable can be considered a container of multiple characters, and hence can be iterated on. Given the following code, answer the questions.

```
str_var = "A string"

count = 0
for c in str_var:
  count += 1

print(count)
```

What is the program's output?

- 7
- 8
- 9

A string variable can be considered a container of multiple characters, and hence can be iterated on. Given the following code, answer the questions.

```
str_var = "A string"

count = 0
for c in str_var:
  count += 1

print(count)
```

What is the program's output?

- [ ] 7
- [x] 8
- [ ] 9

What's the code's output if the line count += 1 is replaced with count *= 2?

```
str_var = "A string"

count = 0
for c in str_var:
  count += 1

print(count)
```

- 0
- 16
- 2^8

What's the code's output if the line count += 1 is replaced with count *= 2?

```
str_var = "A string"

count = 0
for c in str_var:
  count += 1

print(count)
```

- [x] 0
- [ ] 16
- [ ] 2^8

What is printed if the code is changed as follows?

```
str_var = "A string"

count = 0
for c in str_var:
  count += 1
  # New line
  print(c, end = '*')

print(count)
```

- A string*
- A*s*t*r*i*n*g*
- A* *s*t*r*i*n*g*

What is printed if the code is changed as follows?

```
str_var = "A string"

count = 0
for c in str_var:
  count += 1
  # New line
  print(c, end = '*')

print(count)
```

- [ ] A string*
- [ ] A*s*t*r*i*n*g*
- [x] A* *s*t*r*i*n*g*

Range() function in `for` loop

- A `for` loop can be used for iteration and counting.
- The **range()** function is a common approach for implementing counting in a for loop.
- A `range()` function generates a sequence of integers between the two numbers given a step size. This integer sequence is inclusive of the start and exclusive of the end of the sequence.
- The `range()` function can take up to three input values.

  Examples are provided in the following table.

Using the range() function

Range function	Description	Example	Output
`range(end)`	• Generates a sequence beginning at `0` until `end`. • Step size: `1`	`range(4)`	`0, 1, 2, 3`
`range(start, end)`	• Generates a sequence beginning at `start` until `end`. • Step size: `1`	`range(0, 3)`	`0, 1, 2`
		`range(2, 6)`	`2, 3, 4, 5`
		`range(-13, -9)`	`-13, -12, -11, -10`
`range(start, end, step)`	• Generates a sequence beginning at `start` until `end`. • Step size: `step`	`range(0, 4, 1)`	`0, 1, 2, 3`
		`range(1, 7, 2)`	`1, 3, 5`
		`range(3, -2, -1)`	`3, 2, 1, 0, -1`
		`range(10, 0, -4)`	`10, 6, 2`

Two programs printing all multiples of 5 less than 50

Program 1:

```
# For loop condition using
# range() function to print
# all multiples of 5 less than 50
for i in range(0, 50, 5):
  print(i)
```

Program 2:

```
# While loop implementation of printing
# multiples of 5 less than 50
# Initialization
i = 0
# Limiting the range to be less than 50
while i < 50:
    print(i)
    i+=5
```

Quiz

What are the arguments to the `range()` function for the increasing sequence of every 3rd integer from 10 to 22 (inclusive of both ends)?

- `range(10, 23, 3)`
- `range(10, 22, 3)`
- `range(22, 10, -3)`

What are the arguments to the `range()` function for the increasing sequence of every 3rd integer from 10 to 22 (inclusive of both ends)?

- [x] `range(10, 23, 3)`
- [ ] `range(10, 22, 3)`
- [ ] `range(22, 10, -3)`

What are the arguments to the `range()` function for the decreasing sequence of every integer from 5 to 1 (inclusive of both ends)?

- `range(5, 1, 1)`
- `range(5, 1, -1)`
- `range(5, 0, -1)`

What are the arguments to the `range()` function for the decreasing sequence of every integer from 5 to 1 (inclusive of both ends)?

- [ ] `range(5, 1, 1)`
- [ ] `range(5, 1, -1)`
- [x] `range(5, 0, -1)`

What is the sequence generated from `range(-1, -2, -1)`?

- [ ] -1
- [ ] -1, -2
- [ ] -2

What is the sequence generated from `range(-1, -2, -1)`?

- [x] -1
- [ ] -1, -2
- [ ] -2

What is the output of the `range(1, 2, -1)`?

- [ ] 1
- [ ] 1, 2
- [ ] empty sequence

What is the output of the `range(1, 2, -1)`?

- [ ] 1
- [ ] 1, 2
- [x] empty sequence

What is the output of `range(5, 2)`?

- [ ] 0, 2, 4
- [ ] 2, 3, 4
- [ ] empty sequence

What is the output of `range(5, 2)`?

- [ ] 0, 2, 4
- [ ] 2, 3, 4
- [x] empty sequence

TRY IT

5.2 Counting spaces

TRY IT

5.2 Sequences

5.3 Nested Loops

Learning objectives

By the end of this section you should be able to

- Implement nested `while` loops.
- Implement nested `for` loops.

Nested loops

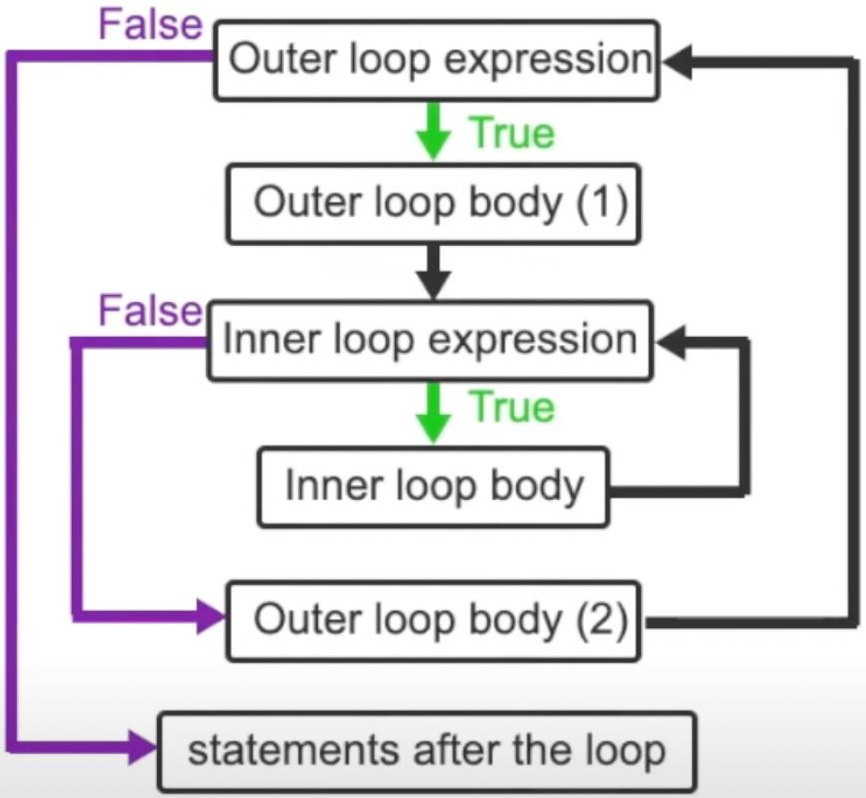
- A **nested loop** has one or more loops within the body of another loop. - The two loops are referred to as **outer loop** and **inner loop**.
- The outer loop controls the number of the inner loop's full execution.
- More than one inner loop can exist in a nested loop.

Nested while loops

Template:

```
while outer_loop_expression:
  # outer loop body (1)
  while inner_loop_expression:
    # inner loop body (2)

# statements after the loop
```



Access for free at https://openstax.org/details/books/introduction-python-programming

Example: Available appointments

- Consider a doctor's office schedule. Each appointment is 30 minutes long. A program to print available appointments can use a nested for loop where the outer loop iterates over the hours, and the inner loop iterates over the minutes.
- This example prints time in hours and minutes in the range between 8:00am and 10:00am.

```
hour = 8
minute = 0
while hour <= 9:
    while minute <= 59:
        print(hour, ":", minute)
        minute += 30
    hour += 1
    minute = 0
```

Quiz

Given the following code, how many times does the print statement execute?

```
i = 1
while i <= 5:
  j = 1
  while i + j <= 5:
    print(i, j)
    j += 1
  i += 1
```

- 5
- 10
- 25

Given the following code, how many times does the print statement execute?

```
i = 1
while i <= 5:
  j = 1
  while i + j <= 5:
    print(i, j)
    j += 1
  i += 1
```

- [ ] 5
- [x] 10
- [ ] 25

What is the output of the following code?

```
i = 1

while i <= 2:
  j = 1
  while j <= 3:
    print('*', end = '')
    j += 1
  print()
  i += 1
```

- ******
- ***
  ***
-
  **
  **
  **

What is the output of the following code?

```
i = 1

while i <= 2:
  j = 1
  while j <= 3:
    print('*', end = '')
    j += 1
  print()
  i += 1
```

- [ ] ******
- [x] ***
  ***
- [ ]
  **
  **
  **

Which program prints the following output?

1 2 3 4

2 4 6 8

3 6 9 12

4 8 12 16

```
i = 1
while i <= 4:
  while j <= 4:
    print(i * j, end = ' ')
    j += 1
  print()
  i += 1
```

```
i = 1
while i <= 4:
  j = 1
  while j <= 4:
    print(i * j, end = ' ')
    j += 1
  print()
  i += 1
```

```
i = 1
while i <= 4:
  j = 1
  while j <= 4:
    print(i * j, end = ' ')
    j += 1
  i += 1
```

Nested for loops

- A nested `for` loop can be used in the same way as a nested while loop.
- A `for` loop is a preferable option in cases where a loop is used for counting purposes using a `range()` function, or when iterating over a container object, including nested situations.
    - Ex: Iterating over multiple course rosters. The outer loop iterates over different courses, and the inner loop iterates over the names in each course roster.

Nested for loops

In the below code, two nested loops are used.

- The outer loop iterates over characters A and B.
- The inner loop iterates over the values 1, 2, and 3.

```
for c in 'AB':
  for i in range(1,4):
    print(c, i, sep='')
```

Mixed loops

- The two `for` and `while` loop constructs can also be mixed in a nested loop construct.
    - Ex: Printing even numbers less than a given number in a list. The outer loop can be implemented using a for loop iterating over the provided list, and the inner loop iterates over all even numbers less than a given number from the list using a while loop.

```
numbers = [12, 5, 3]

i = 0
for n in numbers:
    while i < n:
        print (i, end = " ")
        i += 2
    i = 0
    print()
```

Quiz

Given the following code, how many times does the outer loop execute?

```
for i in range(3):
  for j in range(4):
    print(i, ' ', j)
```

- 3
- 4
- 12

Given the following code, how many times does the outer loop execute?

```
for i in range(3):
  for j in range(4):
    print(i, ' ', j)
```

- [x] 3
- [ ] 4
- [ ] 12

Given the following code, how many times does the inner loop execute?

```
for i in range(3):
  for j in range(4):
    print(i, ' ', j)
```

- [ ] 4
- [ ] 12
- [ ] 20

Given the following code, how many times does the inner loop execute?

```
for i in range(3):
  for j in range(4):
    print(i, ' ', j)
```

- [ ] 4
- [x] 12
- [ ] 20

Which program prints the following output?

0 1 2 3

0 2 4 6

0 3 6 9

```
for i in range(4):
  for j in range(4):
    print(i * j, end = ' ')
  print()
```

```
for i in range(1, 4):
  for j in range(4):
    print(i * j)
```

```
for i in range(1, 4):
  for j in range(4):
    print(i * j, end = ' ')
  print()
```

TRY IT

5.3 Printing a triangle of numbers

TRY IT

5.3 Printing two triangles

5.4 Break and continue

Learning objectives

By the end of this section you should be able to

- Analyze a loop's execution with `break` and `continue` statements.
- Use `break` and `continue` control statements in `while` and `for` loops.

Break

- A **break** statement is used within a `for` or a `while` loop to allow the program execution to exit the loop once a given condition is triggered.
- A `break` statement can be used to improve runtime efficiency when further loop execution is not required.
- Ex: A loop that looks for the character "a" in a given string called `user_string`.

```
user_string = "This is a string."
for i in range(len(user_string)):
    if user_string[i] == 'a':
        print("Found at index:", i)
        break
```

Infinite loop

- A `break` statement is an essential part of a loop that does not have a termination condition.
- A loop without a termination condition is known as an **infinite loop**.
    - Ex: An infinite loop that counts up starting from 1 and prints the counter's value while the counter's value is less than 10. A break condition is triggered when the counter's value is equal to 10, and hence the program execution exits.

```
counter = 1
while True:
    if counter >= 10:
        break
    print(counter)
    counter += 1
```

Quiz

What is the following code's output?

```
string_val = "Hello World"
for c in string_val:
  if c == " ":
    break
  print(c)
```

- Hello
- Hello World
-

H

e

l

l

o

What is the following code's output?

```
string_val = "Hello World"
for c in string_val:
  if c == " ":
    break
  print(c)
```

- ⬜ Hello
- ⬜ Hello World
- ✅

H

e

l

l

o

Given the following code, how many times does the print statement get executed?

```
i = 1
while True:
  if i%3 == 0 and i%5 == 0:
    print(i)
    break
  i += 1
```

- 0
- 1
- 15

Given the following code, how many times does the print statement get executed?

```
i = 1
while True:
  if i%3 == 0 and i%5 == 0:
    print(i)
    break
  i += 1
```

- [ ] 0
- [x] 1
- [ ] 15

What is the final value of i?

```
i = 1
count = 0
while True:
  if i%2 == 0 or i%3 == 0:
    count += 1
  if count >= 5:
    print(i)
    break
  i += 1
```

- [ ] 5
- [ ] 8
- [ ] 30

What is the final value of i?

```
i = 1
count = 0
while True:
  if i%2 == 0 or i%3 == 0:
    count += 1
  if count >= 5:
    print(i)
    break
  i += 1
```

- [ ] 5
- [x] 8
- [ ] 30

Continue

- A `continue` statement allows for skipping the execution of the remainder of the loop without exiting the loop entirely.
- A `continue` statement can be used in a `for` or a `while` loop. - After the `continue` statement's execution, the loop expression will be evaluated again and the loop will continue from the loop's expression.
- A `continue` statement facilitates the loop's control and readability.

Quiz

Given the following code, how many times does the print statement get executed?

```
i = 10
while i >= 0:
  i -= 1
  if i%3 != 0:
    continue
  print(i)
```

- 3
- 4
- 11

Given the following code, how many times does the print statement get executed?

```
i = 10
while i >= 0:
  i -= 1
  if i%3 != 0:
    continue
  print(i)
```

- [ ] 3
- [x] 4
- [ ] 11

What is the following code's output?

```
for c in "hi Ali":
  if c == " ":
    continue
  print(c)
```

h

i

A

l

i

h
i
A
l
i

hi
Ali

TRY IT

5.4 Using break control statement in a loop

TRY IT

5.4 Using a continue statement in a loop

5.5 Loop else

Learning objectives

By the end of this section you should be able to

- Use loop `else` statement to identify when the loop execution is interrupted using a `break` statement.
- Implement a loop `else` statement with a `for` or a `while` loop.

Loop else

- A **loop else** statement runs after the loop's execution is completed without being interrupted by a break statement.
- A loop `else` is used to identify if the loop is terminated normally or the execution is interrupted by a `break` statement.
- Ex: A `for` loop that iterates over a list of numbers to find if the value 10 is in the list.
    - In each iteration, if 10 is observed, the statement "Found 10!" is printed and the execution can terminate using a `break` statement.
    - If 10 is not in the list, the loop terminates when all integers in the list are evaluated, and hence the else statement will run and print "10 is not in the list."
    - Alternatively, a Boolean variable can be used to track whether number 10 is found after loop's execution terminates.

Example: Finding the number 10 in a list

With loop `else`:

```
numbers = [2, 5, 7, 11, 12]
for i in numbers:
  if i == 10:
    print("Found 10!")
    break
else:
  print("10 is not in the list.")
```

Without loop `else`:

```
numbers = [2, 5, 7, 11, 12]
seen = False
for i in numbers:
  if i == 10:
    print("Found 10!")
    seen = True
if seen == False:
  print("10 is not in the list.")
```

Quiz

What is the output?

```
n = 16
exp = 0
i = n
while i > 1:
  if n%2 == 0:
    i = i/2
    exp += 1
  else:
    break
else:
  print(n,"is 2 to the", exp)
```

- [ ] 16 is 2 to the 3
- [ ] 16 is 2 to the 4
- [ ] no output

What is the output?

```
n = 16
exp = 0
i = n
while i > 1:
  if n%2 == 0:
    i = i/2
    exp += 1
  else:
    break
else:
  print(n,"is 2 to the", exp)
```

- [ ] 16 is 2 to the 3
- [x] 16 is 2 to the 4
- [ ] no output

What is the output?

```
n = 7
exp = 0
i = n
while i > 1:
  if n%2 == 0:
    i = i//2
    exp += 1
  else:
    break
else:
  print(n,"is 2 to the", exp)
```

- [ ] no output
- [ ] 7 is 2 to the 3
- [ ] 7 is 2 to the 2

What is the output?

```
n = 7
exp = 0
i = n
while i > 1:
  if n%2 == 0:
    i = i//2
    exp += 1
  else:
    break
else:
  print(n,"is 2 to the", exp)
```

- [x] no output
- [ ] 7 is 2 to the 3
- [ ] 7 is 2 to the 2

What is the output?

```
numbers = [1, 2, 2, 6]
for i in numbers:
  if i >= 5:
    print("Not all numbers are less than 5.")
    break
  else:
    print(i)
    continue
else:
  print("all numbers are less than 5.")
```

1

2

2

6

Not all numbers are less than 5.

1
2
2
Not all numbers are less than 5.

1
2
2
6
all numbers are less than 5.

TRY IT

5.5 Sum of values less than 10

5.6 Summary

Highlights from this module include:

- A `while` loop runs a set of statements, known as the loop body, while a given condition, known as the loop expression, is true.
- A `for` loop can be used to iterate over elements of a container object.
- A `range()` function generates a sequence of integers between the two numbers given a step size.
- A nested loop has one or more loops within the body of another loop.
- A `break` statement is used within a for or a while loop to allow the program execution to exit the loop once a given condition is triggered.
- A `continue` statement allows for skipping the execution of the remainder of the loop without exiting the loop entirely.
- A loop `else` statement runs after the loop's execution is completed without being interrupted by a break statement.

TRY IT

5.6 Prime numbers